



Claude Code 하네스 엔지니어링 입문

.claude 폴더에 에이전트와 스킬을 만들어
나만의 AI 작업 환경 구축하기

초보자를 위한 스텝 바이 스텝 강의 · 2026 년 7 월

하네스 엔지니어링이란 ?



harness = 마구 (馬具)

말이 아무리 힘이 세도
마구가 없으면 그 힘으로
마차를 끌 수 없다 .

강력한 AI 모델 (말) 이 내 프로젝트에서
원하는 방식으로 일하도록 **주변 환경 (마
구)** 을 설계하는 것

프롬프트 엔지니어링 — 매번 대화할 때 무엇을
말할지 고민

하네스 엔지니어링 — 말하지 않아도 우리 팀
방식대로 일하게 만드는 구조를 한 번만 구축

Claude Code 에서 그 하네스는 **.claude**
폴더 입니다

오늘 이 강의에서 만들 것



스킬 1 개 — **commit-helper**

커밋 메시지를 우리 팀 규칙대로 작성하는 방법을 Claude 에게 가르치는 지식 패키지



에이전트 1 개 — **code-reviewer**

코드 변경 사항을 격리된 공간에서 자동 검토하고 결과만 보고하는 전문 작업자

완성하면 ? " 커밋해 줘 " → 팀 규칙대로 커밋 / " 리뷰해 줘 " → 전문 에이전트가 자동 검토

스킬 vs 에이전트 — 레시피와 요리사

구분	스킬 (Skill) 📖 레시피	에이전트 (Subagent) 👨‍🍳 요리사
무엇	작업 절차 · 노하우를 담은 지식 패키지	독립 컨텍스트에서 일하는 전문 작업자
위치	.claude/skills/ 이름 /SKILL.md	.claude/agents/ 이름 .md
실행	현재 대화에 지침이 로드됨	별도 공간에서 실행 → 결과만 반환
용도	반복 규칙 · 체크리스트 전수	코드 리뷰 등 격리가 유리한 작업

레시피 (스킬) 는 지금 일하는 사람이 읽고 따라 하는 것 , **요리사 (에이전트)** 는 일을 통째로 맡기면 자기 작업대에서 완성해 오는 존재입니다 .

사전 준비 — 3 가지만 확인하세요



① Claude Code 설치

Node.js 18 이상 필요



② 로그인

claude 실행 → Pro/Max 구독 또는 API 키



③ Git 확인

git --version 으로 설치 확인

터미널

설치

```
npm install -g @anthropic-ai/claude-code
```

설치 확인

```
claude --version
```



버전 번호가 출력되면 준비 완료 !

STEP 1

연습용 프로젝트와 .claude 폴더 만들기

터미널

```
mkdir my-harness-demo  
cd my-harness-demo  
git init
```

하네스의 뼈대

```
mkdir -p .claude/agents  
mkdir -p .claude/skills/commit-helper
```

완성된 폴더 구조

```
my-harness-demo/  
└─ .claude/  
    └─ agents/  
        └─ ( 에이전트 .md 파일 )  
    └─ skills/  
        └─ commit-helper/  
            └─ SKILL.md
```

규칙 2 가지 : 에이전트는 agents/ 이름 .md 파일 하나 , 스킬은 폴더 안에 반드시 **SKILL.md** 라는 이름으로 !

첫 번째 스킬 만들기 — commit-helper

```
.claude/skills/commit-helper/SKILL.md
---
name: commit-helper
description: git 커밋 메시지를 작성할 때 사용 .
  사용자가 커밋 , commit, 변경사항 저장을
  요청하면 이 규칙에 따라 작성한다 .
---
# 커밋 메시지 작성 규칙
1. git diff 로 변경 내용을 먼저 파악한다 .
2. 형식 : <타입> : <50자 이내 한글 요약>
   타입 : feat / fix / docs / refactor / chore
3. 성격이 다른 변경은 커밋 분리를 제안한다 .
4. 커밋 전 메시지를 보여주고 확인받는다 .
```

스킬 = 파일 하나

위쪽 --- 부분은 **프런트매터** (메타데이터)

아래 본문은 Claude 가 따라 할 작업 절차

메모장으로도 OK!

왼쪽 내용을 그대로 복사해 파일로 저장하면 끝입니다 .

description 은 소개글이 아니라 '방아쇠'



Claude 는 시작할 때 모든 스킬의 이름과 description 만 읽어 둡니다 .
대화가 **description** 과 맞아떨어지는 순간 본문 전체를 불러옵니다 .

✗ 발동 안 됨

description: 커밋 도우미

짧고 모호함 — 언제 써야 할지 알 수 없음

✓ 잘 발동됨

description: git 커밋 메시지를
작성할 때 사용 . 사용자가 커밋 ,
commit 을 요청하면 ...

무엇을 + 언제 , 실제 쓸 법한 키워드 포함

공식 : " 무엇을 하는 스킬인지 + 언제 사용해야 하는지 " 를 사용자의 언어로 쓰기

첫 번째 에이전트 만들기 — code-reviewer

```
.claude/agents/code-reviewer.md
```

```
---
```

```
name: code-reviewer
```

```
description: 코드 변경 사항을 검토한다 . 사용자가  
코드 리뷰 , 검토 , review 를 요청할 때 사용한다 .
```

```
tools: Read, Grep, Glob, Bash
```

```
---
```

```
당신은 꼼꼼한 시니어 코드 리뷰어입니다 .
```

1. git diff HEAD 로 최근 변경을 확인한다 .
2. 버그 · 보안 · 가독성 관점에서 문제를 찾는다 .
3. 심각도 (높음 / 중간 / 낮음) 와 파일 : 줄번호로 보고 .
4. 각 문제의 최소 수정 방법을 제안한다 .

```
중요 : 파일을 수정하지 말고 보고서만 작성 .
```

에이전트 = 별도의 Claude

자기만의 컨텍스트 , 시스템 프롬프트 ,
도구 권한을 가진 독립 작업자

본문 = 시스템 프롬프트

에이전트는 메인 대화를 못 봅니다 .
그래서 1 번이 스스로 git diff 를
실행하는 절차인 것 !

" 하지 마 " 라고 부탁하지 말고 , 할 수 없게 설계하라



허용된 도구

Read · Grep · Glob · Bash

읽기 , 검색 , 명령 실행



주지 않은 도구

Write · Edit

리뷰어가 실수로 코드를 고치는 일이 구조적으로 불가능

tools 항목 **한 줄**이 하네스 엔지니어링의 핵심 사고방식을 보여줍니다 .

STEP 4

테스트 ① — 에이전트가 버그를 잡는가 ?

터미널 — 일부러 문제 있는 코드 만들기

```
cat > app.py << 'EOF'
API_KEY = "sk-secret-1234"
        # 문제 1: 키 하드코딩
def divide(a, b):
    return a / b
        # 문제 2: 0 나누기 미처리
EOF
```

Claude Code 실행 후 대화

```
$ claude

> /agents
  (code-reviewer 확인 )

> app.py 코드 리뷰해 줘
```



성공 기준 : " 코드 리뷰 " 라는 말만으로 code-reviewer 가 자동 위임되고 , API 키 하드코딩 (높음) 과 0 나누기 (중간) 를 지적한 보고서가 돌아온다 . 자동 위임이 안 되면 이름을 직접 불러도 OK.

테스트 ② — 스킬이 커밋 규칙을 지키는가 ?

Claude Code 대화창

> 지적된 문제 두 가지를 고쳐 줘

> 좋아, 이제 커밋해 줘

(commit-helper 스킬 자동 발동)

Claude 가 제안하는 커밋 메시지

fix: API 키 하드코딩 제거
및 0 나누기 예외 처리

이대로 커밋할까요? (확인 요청)

 규칙을 매번 말하지 않았는데도 지켜진다면 —

여러분의 첫 번째 하네스가 완성된 것입니다.

동작 원리 — 점진적 공개 (Progressive Disclosure)



평소엔 이름 + description 만 메모리에 . **필요한 순간에만 본문 로드** .
→ 스킬을 수십 개 설치해도 부담 없음 . description 품질이 하네스의 8 할 !

자주 겪는 문제 4 가지



발동이 안 돼요

description 이 모호함 → 실제 쓸 단어 (커밋 , 리뷰) 를
넣고 " ~ 할 때 사용 " 문장 추가



새 에이전트가 목록에 없어요

에이전트는 세션 시작 시 로드 → Claude Code 재시작
후 /agents 로 확인



SKILL.md 인식이 안 돼요

파일명이 정확히 SKILL.md 인지 , --- 가 첫 줄인지 ,
YAML 콜론 뒤 공백 확인



에이전트가 맥락을 몰라요

정상 ! 메인 대화를 못 봄 → 위임 시 정보를 함께
전달하거나 자체 수집 절차 작성

NEXT STEPS

다음 단계로 나아가기



/agents 명령

대화형으로 에이전트 초안 자동 생성



개인용 하네스

~/.claude/ 에 두면 모든 프로젝트 공통



스크립트 동봉

스킬 폴더에 검증된 스크립트를 함께



모델 지정

model: haiku 로 탐색 작업 비용 절감



팀과 공유

.claude 를 git 커밋 → 팀 작업
표준으로



공식 문서

code.claude.com/docs 의
Subagents · Skills

하네스는 한 번 만들면 팀 전체의 작업 방식이 됩니다. 오늘 만든 두 파일이 그 시작입니다. 🐎